Links

GitHub repository: https://github.com/nylandgit/IND320-Repo

Streamlit app: https://nylandgit-ind320-repo.streamlit.app/

Code runthrough video: https://youtu.be/F2-y1frza1k

AI Statement and Descriptive Log

AI Statement

This project has used the free VS Code Copilot generative AI to: Generate code, identify bugs, fix bugs, brainstorm technical solutions, and to teach the process of coding through in-depth explanations of each generated line of code. Some Google searches have also involved Google's built-in generative AI.

The project author has steered the AI's coding through accepting, rejecting, and modifying the AI's code based on the author's intuition. The author has requested code explanations, studied the AI's code, and made explanatory comments for each code block based on the author's understanding.

The code provided is therefore created partly by the author and partly by generative AI. The explanatory code comments are written exclusively by the author.

Descriptive Log

The project was done in two parts: First, the csv data was loaded and processed locally using Pandas in this notebook. Second, an online Streamlit app was creating, using the same csv data to create a dashboard. In the second part, the data processing done in part one was translated to be Streamlit-compatible. The work was structured largely as the contents of this file are structured, meaning "Lead Packages and Data" was (for instance) done before "Translate Norwegian Columns", etc. This log was not written until the project was finished. Therefore, we consider its value as a "log" to be negligible.

-- Part 1.1 --

The csv data was processed as described in section 1.1 of this file.

Notes of interest: Plotting "all" columns together and "each" column separately was considered unnatural by the author, as this would involve plotting weeks/time, constant values, etc. Therefore, a subset of columns were chosen, and from these, only data from Area Type = EL and Area Number = 1 was used. Further selecting and filtering of data was intented, but later abandoned, as the author considers this to be unnecessary until further assignments are made available, and the author knows what data in specific is of interest.

-- Part 1.2 --

The Streamlit app was created using app.py under ...\Oblig. The custom module db.py under ...\Oblig\modules was created to do a number of initial csv-related tasks common for the app's two pages, 1_reservoirs_data.py and 2_data_plots.py, located under ...\Oblig\pages. The code in all aforementioned files is copied and pasted into this file.

For this analysis, non-ratio data (filling TWh) was considered to be normalized, as to make simultaneous plotting more natural, but it was abandoned as the figure looks okay enough without, and the author does not yet know which data columns in specific will be relevant for the future of the project.

1.1 Resovoir Data Analysis

Load Packages and Data

```python
import pandas as pd
import matplotlib.pyplot as plt

# Start by reading the CSV file from the path into a DataFrame.
df = pd.read_csv(r"../data/reservoirs.csv")
```

Translate Norwegian Columns

```python
# Translating Norwegian column names into understandable English.
df = df.rename(columns={
    "dato_Id": "Date",
    "omrType": "Area Type",
    "omrnr": "Area Number",
    "iso_aar": "ISO Year",
    "iso_uke": "ISO Week",
    "fyllingsgrad": "Filling Ratio",
    "kapasitet_TWh": "Capacity TWh",
    "fylling_TWh": "Filling TWh",
    "neste_Publiseringsdato": "Next Publishing Date",
    "fyllingsgrad_forrige_uke": "FR Last Week",
    "endring_fyllingsgrad": "Change FR",
})
```

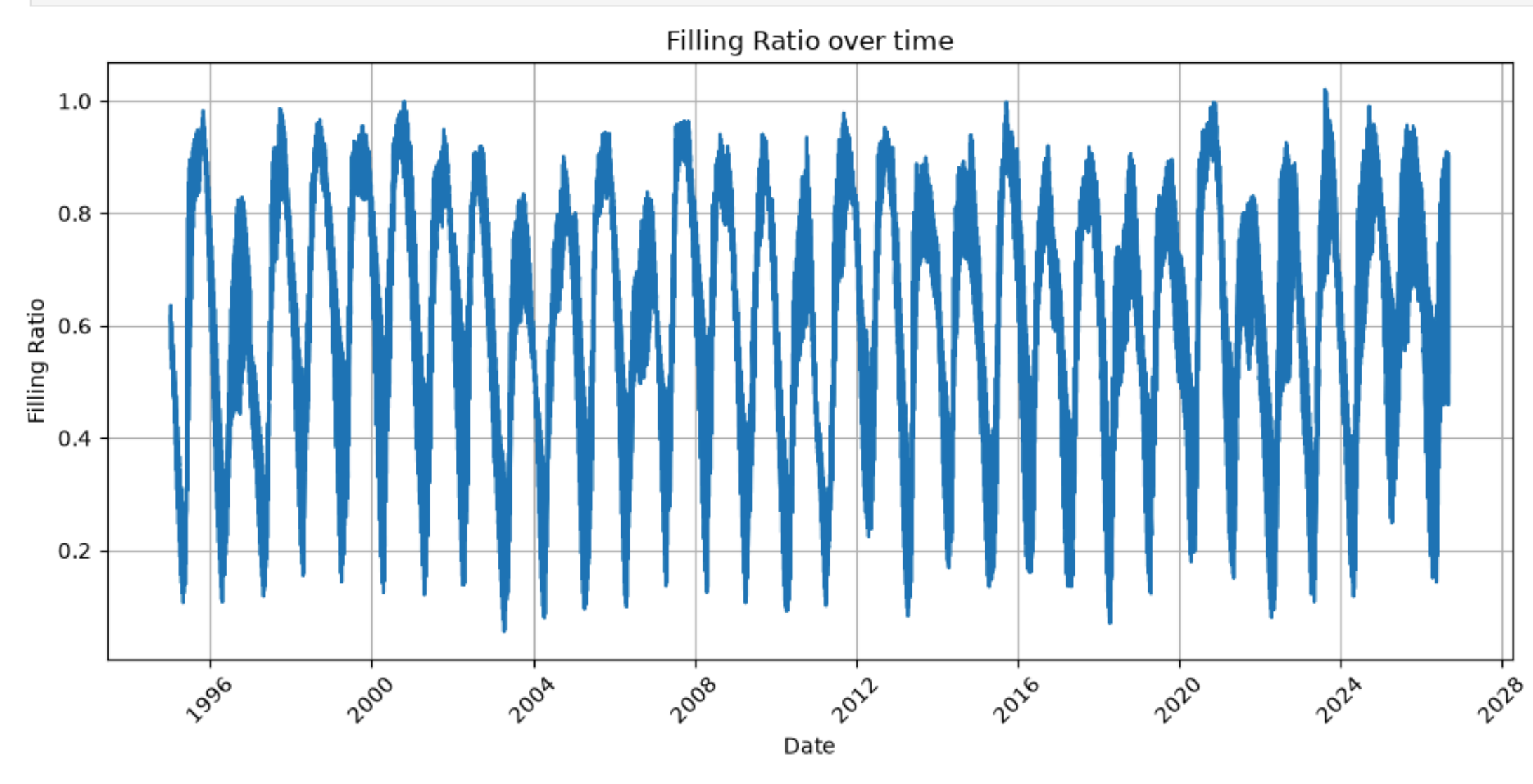
Sort Columns by Date

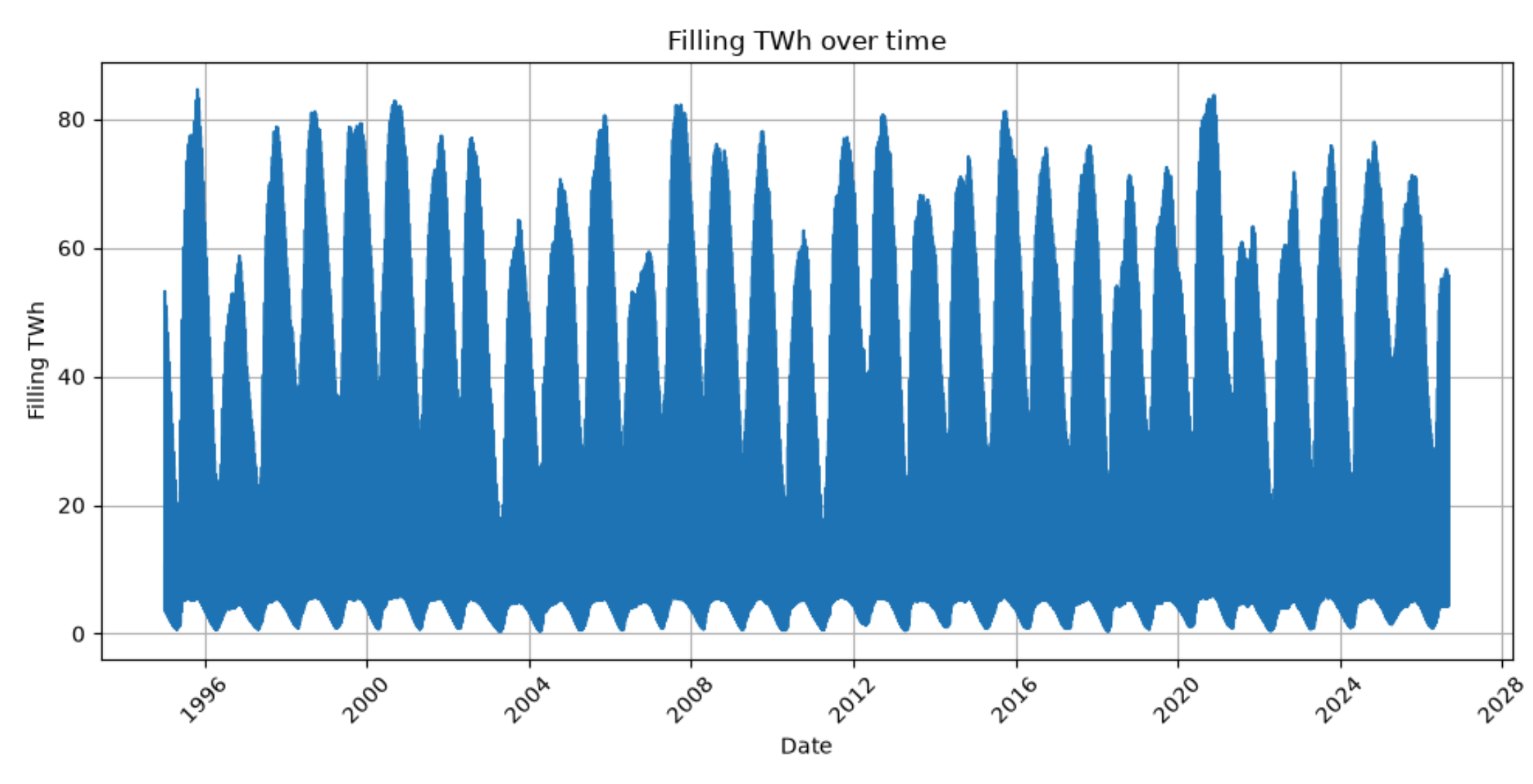
```python
# Setting up sorting by date.
df["Date"] = pd.to_datetime(df["Date"])
df = df.sort_values("Date", ascending=True)

# Optionally, user input can be chosen to dictate sorting.
"""
# Sorting by user input.
sort = input("Welcome, User\n"
            "Select sorting column:\n" \
            "1 - Date (oldest-newest)\n" \
            "2 - Date (newest-oldest)")

if sort == "1":
    df = df.sort_values("Date", ascending=True)
elif sort == "2":
    df = df.sort_values("Date", ascending=False)
else:
    print("Invalid choice.")
"""
```

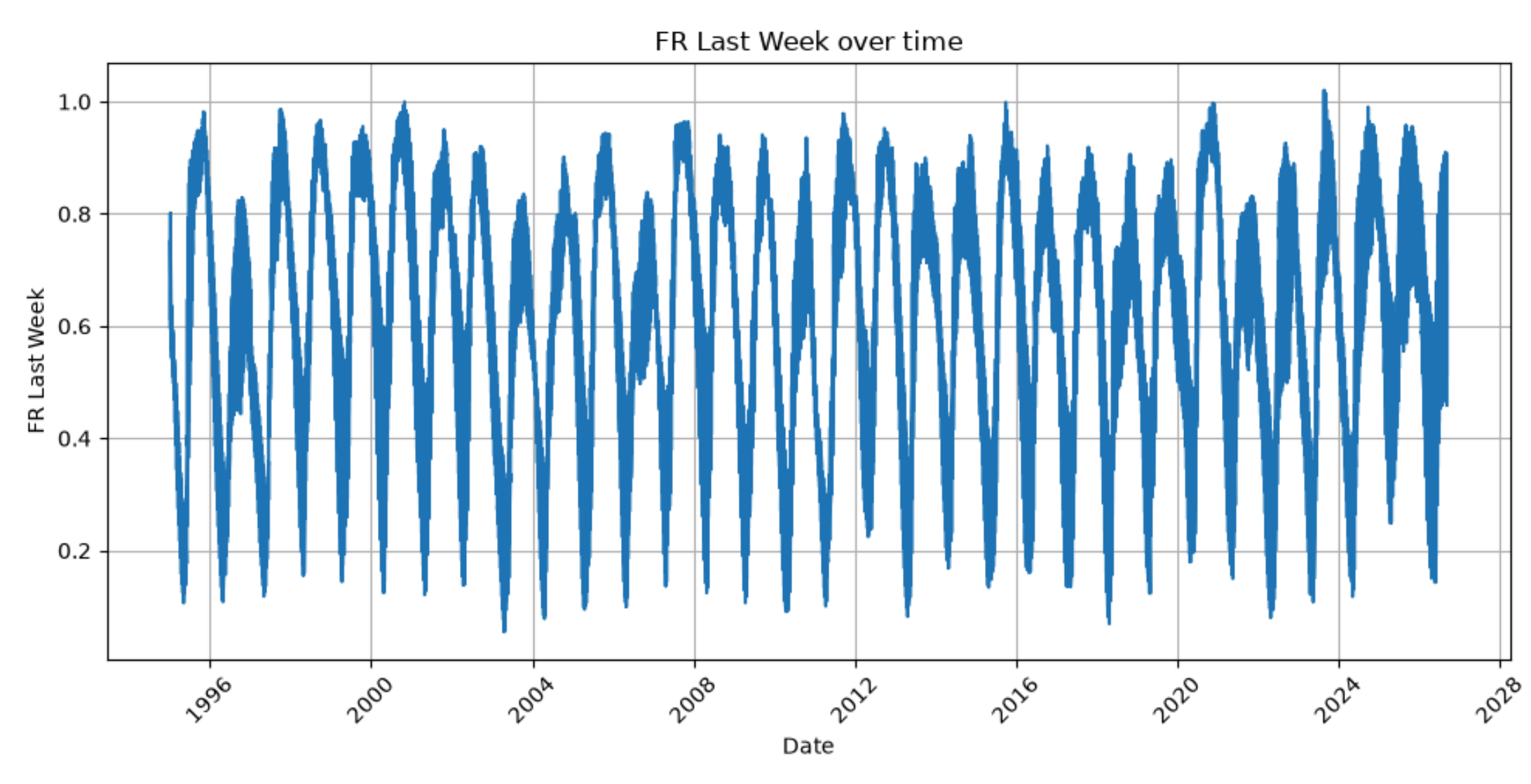
```
Out[17]: '\n# Sorting by user input.\nsort = input("Welcome, User\n"\n            "Select sorting column:\n"             "1 - Date (oldest-newest)\n"             "2 - Date (newest-oldest)")\n\nif sort == "1":\n    df = df.sort_values("Date", ascending=True)\nelif sort == "2":\n    df = df.sort_values("Date", ascending=False)\nelse:\n    print("Invalid choice.")\n'
```

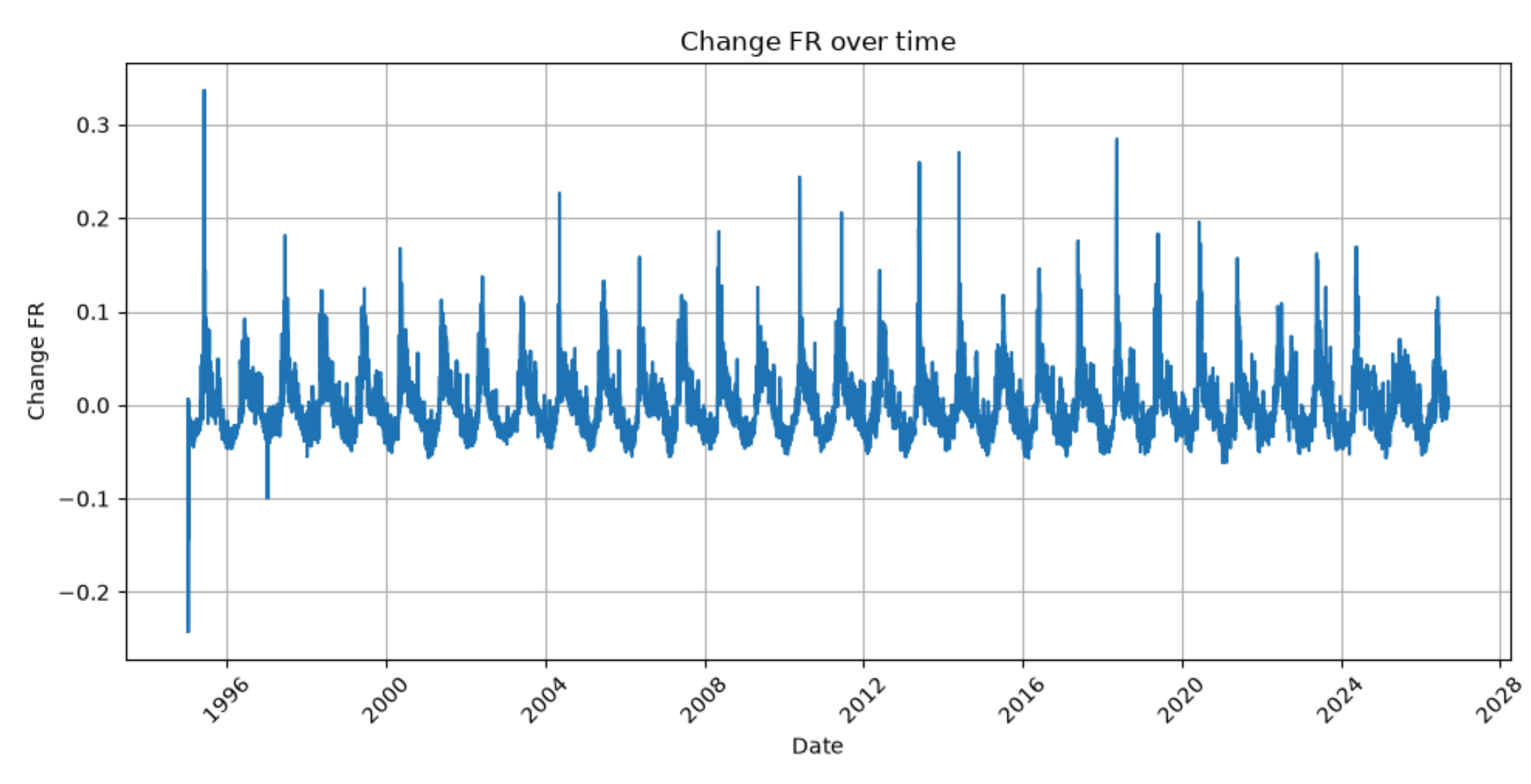
Plotting Columns Individually

```python
# Picking out columns to plot individually.
columns_to_plot = [
    "Filling Ratio",
    "Filling TWh",
    "FR Last Week",
    "Change FR",
]

# Plotting selected columns for individual plotting.
for column in columns_to_plot:
    plt.figure(figsize=(10, 5))
    plt.plot(df["Date"], df[column])

    plt.title(f"{column} over time")
    plt.xlabel("Date")
    plt.ylabel(column)
    plt.xticks(rotation=45)
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```



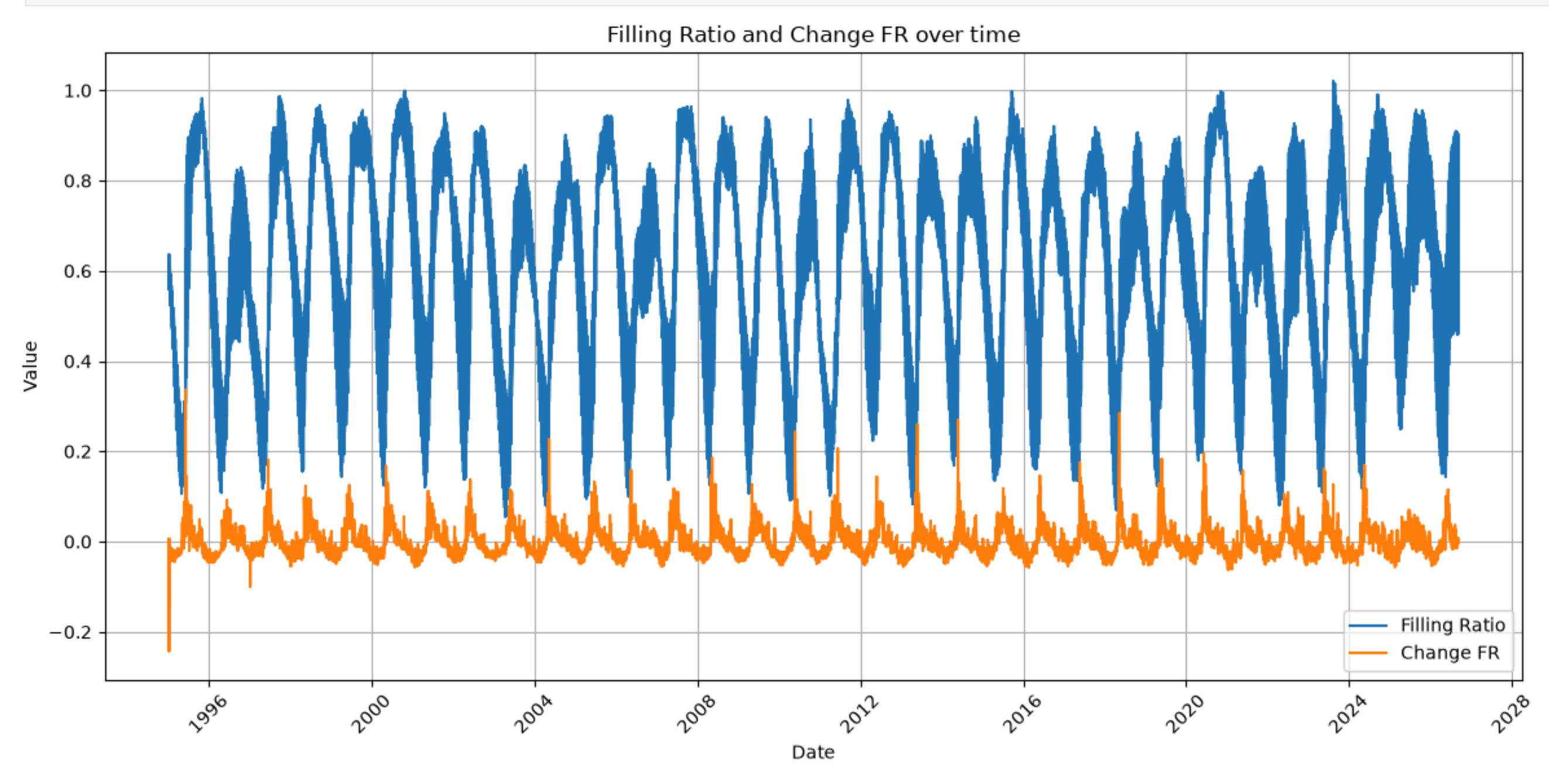






Plotting Columns Together

```python
# Plot Filling Ratio and Change FR together.
plt.figure(figsize=(12, 6))
plt.plot(df["Date"], df["Filling Ratio"], label="Filling Ratio")
plt.plot(df["Date"], df["Change FR"], label="Change FR")

plt.title("Filling Ratio and Change FR over time")
plt.xlabel("Date")
plt.ylabel("Value")
plt.xticks(rotation=45)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```



Print Data Excerpt

```python
print(df.head())
print(df.tail())
```

```
           Date Area Type  Area Number  ISO Year  ISO Week  Filling Ratio  \
3468 1995-01-08        EL            5      1995         1       0.616448
3361 1995-01-08        EL            4      1995         1       0.558864
5036 1995-01-08        EL            3      1995         1       0.602376
3113 1995-01-08        EL            1      1995         1       0.621467
7739 1995-01-08        EL            2      1995         1       0.630947

      Capacity TWh  Filling TWh Next Publishing Date  FR Last Week  Change FR
3468     17.390587    10.720388  0001-01-01T00:00:00      0.614288   0.002160
3361     21.079208    11.780413  0001-01-01T00:00:00      0.801348  -0.242484
5036      8.920932     5.373754  0001-01-01T00:00:00      0.596027   0.006349
3113      6.003264     3.730831  0001-01-01T00:00:00      0.734888  -0.113421
7739     34.043590    21.479713  0001-01-01T00:00:00      0.777604  -0.146657
            Date Area Type  Area Number  ISO Year  ISO Week  Filling Ratio  \
8263  2026-09-06        EL            4      2026        36       0.902762
14875 2026-09-06      VASS            1      2026        36       0.570397
14873 2026-09-06      VASS            3      2026        36       0.826252
8264  2026-09-06        EL            2      2026        36       0.460863
14876 2026-09-06      VASS            2      2026        36       0.516978

       Capacity TWh  Filling TWh Next Publishing Date  FR Last Week  Change FR
8263      21.079208    19.029512  2026-09-16T13:00:00      0.906918  -0.004156
14875     36.044464    20.559645  2026-09-16T13:00:00      0.567087   0.003310
14873     28.155403    23.263466  2026-09-16T13:00:00      0.827641  -0.001389
8264      34.043590    15.689445  2026-09-16T13:00:00      0.458999   0.001865
14876     23.237715    12.013381  2026-09-16T13:00:00      0.514423   0.002555
```

1.2 Streamlit code copy

1.2.1 CSV loading and caching module, translation, sorting

```python
from pathlib import Path
import pandas as pd
import streamlit as st

# Defining .csv path relative to data page .py file.
DATA_PATH = Path(__file__).parent.parent / "data" / "reservoirs.csv"

# Instructing Streamlit to cache load data function so the csv isn't
# reloaded with every Streamlit page action.
@st.cache_data
def load_data():
    df = pd.read_csv(DATA_PATH)

    # Translating Norwegian column names into understandable English.
    df = df.rename(columns={
        "dato_Id": "Date",
        "omrType": "Area Type",
        "omrnr": "Area Number",
        "iso_aar": "ISO Year",
        "iso_uke": "ISO Week",
        "fyllingsgrad": "Filling Ratio",
        "kapasitet_TWh": "Capacity TWh",
        "fylling_TWh": "Filling TWh",
        "neste_Publiseringsdato": "Next Publishing Date",
        "fyllingsgrad_forrige_uke": "FR Last Week",
        "endring_fyllingsgrad": "Change FR",
    })

    # Sorting by date, oldest to newest.
    df["Date"] = pd.to_datetime(df["Date"])
    return df.sort_values("Date", ascending=True)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[21], line 6
      2 import pandas as pd
      3 import streamlit as st
      4 
      5 # Defining .csv path relative to data page .py file.
----> 6 DATA_PATH = Path(__file__).parent.parent / "data" / "reservoirs.csv"
      7 
      8 # Instructing Streamlit to cache load data function so the csv isn't
      9 # reloaded with every Streamlit page action.

NameError: name '__file__' is not defined
```

1.2.2 Data page: Showing imported data, LineChartColumn().

Loading CSV

```python
from modules.db import DATA_PATH, load_data
import pandas as pd
import streamlit as st

# Loading csv into df.
df = load_data()

# Creating page title and csv load information.
st.title("Reservoir data")
st.write(f"Loaded {len(df):,} rows from {DATA_PATH.name}.")
```

Plotting w/ LineChartColumn()

```python
# Selecting csv columns to include in chart. Plotting ISO weeks/years
# is considered unnecessary.
columns_to_plot = [
    "Filling Ratio",
    "Filling TWh",
    "FR Last Week",
    "Change FR",
]

# Selecting data from specific area type and number.
plot_area_type = "EL"
plot_area_number = 1

# Selecting data from df matching specified area type and number.
# Plotting all data in chart will lead to overlapping dates, as
# multiple area types and numbers share the same dates.
plot_data = df[
    (df["Area Type"] == plot_area_type)
    & (df["Area Number"] == plot_area_number)
][["Date", *columns_to_plot]].set_index("Date")

# Creating chart title and information.
st.write(
    f"Showing {plot_area_type} area {plot_area_number} "
    f"from {plot_data.index.min().date()} to {plot_data.index.max().date()}."
)

# Creating two-column table w/ column titles and LineChartColumn().
chart_table = pd.DataFrame(
    {
        "Data category": columns_to_plot,
        "Flowchart": [plot_data[column].dropna().tolist() for column in columns_to_plot],
    }
)
st.dataframe(
    chart_table,
    column_config={
        "Flowchart": st.column_config.LineChartColumn("Flowchart"),
    },
    hide_index=True,
    use_container_width=True,
)
```

1.2.3 Plotting page

Load CSV

```python
from modules.db import load_data
import pandas as pd
import streamlit as st

# Loading csv into df.
df = load_data()

# Creating page title and csv load information.
st.title("Data Plots")
st.write(f"Select data type and range to plot.")
```

Plotting Setup

```python
# Selecting csv columns to include in chart. Plotting ISO weeks/years
# is considered unnecessary.
columns_to_plot = [
    "Filling Ratio",
    "Filling TWh",
    "FR Last Week",
    "Change FR",
]

# Selecting data from specific area type and number.
plot_area_type = "EL"
plot_area_number = 1

# Selecting data from df matching specified area type and number.
# Plotting all data in chart will lead to overlapping dates, as
# multiple area types and numbers share the same dates.
plot_data = df[
    (df["Area Type"] == plot_area_type)
    & (df["Area Number"] == plot_area_number)
][["Date", *columns_to_plot]].set_index("Date")
```

Selection Slider

```python
# Preparing selectable months for selection slider using list:
# * plot_data.index selects the index values (Date) for each column
#   in plot_data. These dates are already sorted chronologically.
# * .to_period("M") converts the dates into monthly periods.
# * .unique() makes the months show up only once in the slider.
# * .tolist() makes available_months a Python list. We now have a de-
#    duplicated, ordered list of available months for our slider.
available_months = plot_data.index.to_period("M").unique().tolist()

# Creating selection slider for selecting specific subsets of months,
# using ready-made available_months list.
start_month, end_month = st.select_slider(
    "Select months",
    options=available_months,
    value=(available_months[0], available_months[0]),
    format_func=str,
)

# Creating pandas object for plotting monthly data in figures. Some
# months in the CSV have more observations than others.
plot_months = plot_data.index.to_period("M")

# Selecting months from plot_months based on selection slider input.
selected_data = plot_data.loc[
    (plot_months >= start_month) & (plot_months <= end_month)
]
```

Selection Box

```python
# Creating selection box.
selected_column = st.selectbox(
    "Column to plot",
    options=["All columns", *columns_to_plot],
)

# Selecting column(s) to plot based on selection box input, default
# "All columns".
selected_columns = (
    columns_to_plot if selected_column == "All columns" else [selected_column]
)
```

Plotting

```python
# Creating plot based on selected colum(s) and data.
st.write(
    f"Showing {plot_area_type} area {plot_area_number} "
    f"from {selected_data.index.min().date()} to {selected_data.index.max().date()}."
)
st.subheader("Reservoir metrics over time")
st.line_chart(
    selected_data,
    y=selected_columns,
    x_label="Date",
    y_label="Metric value",
)
```